■ Firebase Cloud Messaging

Complete Developer Guide

Architecture • End-to-End Flow • Android Integration • Code Samples

Topics Covered

Android • Kotlin • Firebase SDK • 2024 / 2025

1 What is Firebase Cloud Messaging (FCM)?

Firebase Cloud Messaging (FCM) is a cross-platform messaging solution from Google that allows you to reliably deliver messages and notifications to Android, iOS, and web applications at no cost. It is the successor to Google Cloud Messaging (GCM) and is part of the Firebase platform.

FCM acts as a reliable intermediary between your application server and client devices. Rather than maintaining persistent connections to millions of devices yourself, you delegate that complexity to Google's infrastructure, which handles delivery, queuing, retries, and platform-specific transport layers.

Key Capabilities

Capability	Description
Notification Messages	Displayed automatically by the FCM SDK in the device notification tray
Data Messages	Silent payloads delivered to your app for custom processing
Topic Messaging	Broadcast to all devices subscribed to a named topic
Device Group Messaging	Send to a logical group of devices belonging to one user
Upstream Messaging	Send messages from device back to your app server (XMPP only)
Analytics Integration	Built-in delivery and open-rate tracking via Firebase Analytics

Message Types

FCM supports two fundamental message types that can be combined:

Notification Message — Has a predefined set of user-visible keys (title, body, icon, sound, click_action). When the app is in the background, FCM SDK displays it automatically without any app code running. This is the most common type for user-facing alerts.

Data Message — Contains only custom key-value pairs. Always delivered to the app's onMessageReceived() callback regardless of app state (foreground or background). Used for silent updates, syncing data, or custom notification rendering.

Combined Message — Contains both notification and data payloads. In the foreground onMessageReceived() fires; in the background the notification is auto-displayed and data is available via extras in the launch Intent.

2 Architecture of FCM

FCM's architecture consists of three primary layers that work together to deliver messages from your server to a user's device reliably and efficiently.

The Three-Layer Architecture

Layer	Component	Responsibility
Layer 1 — Sender	Your App Server / Backend	Creates and sends message payloads to FCM using HTTP v1 A
Layer 1 — Sender	Firebase Admin SDK	Server-side library (Node.js, Java, Python, Go, C#) abstracting F
Layer 2 — FCM Backend	FCM Connection Servers	Accepts messages from your server, queues them, handles fan-
Layer 2 — FCM Backend	Transport Layer	Routes to correct platform transport: APNs (iOS), FCM Android t
Layer 3 — Client	FCM SDK (on device)	Maintains persistent connection, receives messages, invokes ap
Layer 3 — Client	Your Android App	Handles messages via FirebaseMessagingService, manages FC

Core Components Explained

FCM Registration Token

Every app instance receives a unique FCM Registration Token — a long string that identifies a specific app on a specific device. Think of it as a precise delivery address. Your server must store this token to send messages to that particular user/device.

IMPORTANT
Tokens can change: after app reinstall, device restore, data clear, or token expiry. Always update your server when onNewToken() fires in FirebaseMessagingService.

FCM Connection Servers

Google operates a globally distributed set of FCM connection servers. They accept messages from your backend, validate credentials, apply rate limiting, queue messages for offline devices (up to 4 weeks for collapsible messages), and route to the appropriate platform transport. They also handle message collapse (replacing old undelivered messages with newer ones).

Platform Transport Layers

Platform	Transport Used	Notes
Android	FCM Android Transport	Direct persistent connection maintained by Google Play Services
iOS	APNs (Apple Push)	FCM wraps the message and forwards to Apple's APNs infrastructure
Web	Web Push Protocol	Uses browser-level push infrastructure (Chrome, Firefox, etc.)

Message Lifecycle States

State	Meaning
Accepted	FCM has received the message from your server

State	Meaning
Queued	Device is offline; message is queued (TTL applies)
Delivered	FCM transport delivered to device
Pending	App has not yet acknowledged receipt
Expired	TTL elapsed without delivery; message discarded

3 End-to-End Flow for FCM

Complete Message Journey

The following steps trace the complete lifecycle from a user action on your backend all the way to the notification appearing on the user's screen.

Step 1	**App Registration** On first launch, the FCM SDK running inside Google Play Services generates a unique Registration Token for the app instance. FirebaseMessagingService.onNewToken() is called in your app with this token.
Step 2	**Token Storage** Your app sends the token to your backend server via your own API. The server stores it associated with the user account. This is a critical step — without the token, your server cannot address messages to this device.
Step 3	**Trigger on Server** A business event occurs (new chat message, order shipped, breaking news). Your backend constructs an FCM message payload containing the registration token(s) as the target and a notification/data body.
Step 4	**HTTP v1 API Call** Your server sends an HTTPS POST request to https://fcm.googleapis.com/v1/projects/{project_id}/messages:send with an OAuth 2.0 bearer token in the Authorization header and the message JSON in the body.
Step 5	**FCM Validation & Queuing** FCM connection servers validate credentials, check message format and size limits (4KB for data, defined limits for notification keys), then either deliver immediately if the device is online, or queue the message if offline (respecting the TTL value).
Step 6	**Transport to Device** FCM uses its persistent connection to Google Play Services on the Android device to push the message payload. This connection is maintained system-wide — not per app — making it extremely battery efficient.
Step 7	**On-Device Routing** Google Play Services receives the message and routes it to the correct app based on the package name embedded in the registration token.
Step 8	**App Processing** If the app is in the FOREGROUND: onMessageReceived() fires. Your code handles display. If the app is in the BACKGROUND / KILLED: FCM SDK auto-displays notification messages. Data-only messages are queued until the app next opens (or delivered via high-priority).

Step 9	**User Interaction** The user taps the notification. The system opens your launcher Activity (or the activity specified in click_action). The original FCM data payload is available in getIntent().getExtras().

4 How FCM Works with Android

Android's FCM integration is built on top of Google Play Services, which maintains a single persistent connection per device — regardless of how many apps use FCM. This architecture is battery-friendly since one connection serves all apps.

Setup & Dependencies

1. Add google-services.json to your app module (download from Firebase Console).
2. Apply the Google Services plugin in your build files.
3. Add the FCM dependency to your module's build.gradle.kts.

```
// Project-level build.gradle.kts
plugins {
    id("com.google.gms.google-services") version "4.4.0" apply false
}

// App-level build.gradle.kts
plugins {
    id("com.google.gms.google-services")
}
dependencies {
    implementation(platform("com.google.firebase:firebase-bom:32.7.0"))
    implementation("com.google.firebase:firebase-messaging-ktx")
}
```

Foreground vs Background Behaviour

App State	Notification Message	Data Message
Foreground	onMessageReceived() fires — YOU must display notification	onMessageReceived() fires — you handle payload
Background	FCM SDK auto-displays in notification tray	onMessageReceived() fires (high priority) OR queued
Killed / Not running	FCM SDK auto-displays; data in Intent extras on tap	Delivered when app next comes to foreground

Notification Channels (Android 8.0+)

From Android 8.0 (API 26) onwards, all notifications MUST be assigned to a Notification Channel. Channels let users control notification behaviour per category (sound, vibration, importance). You must create your channels before posting notifications — typically in Application.onCreate().

REQUIRED
Notifications posted without a valid channel ID are silently dropped on Android 8.0+. Always create your channel in Application.onCreate() before the first notification arrives.

FCM Token Lifecycle in Android

Event	Token Behaviour
App first install	New token generated; onNewToken() called
App data cleared	Token invalidated; new token generated
App reinstalled	Token invalidated; new token generated
Token rotated by FCM	onNewToken() called with new token — must update server
Device factory reset	All tokens invalidated

Permissions

On Android 13 (API 33) and above, you must explicitly request the POST_NOTIFICATIONS runtime permission. Without it, no notifications will appear.

```
// AndroidManifest.xml
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>

// In your Activity (Android 13+)
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
    requestPermissions(arrayOf(Manifest.permission.POST_NOTIFICATIONS), 1001)
}
```

5 Complete Code Sample — Order Shipped Alert

The following example demonstrates a real-world scenario: an e-commerce app that sends a push notification when a user's order has shipped. Every line is annotated.

Step 1 — FirebaseMessagingService (Kotlin)

Create a class that extends FirebaseMessagingService. Declare it in AndroidManifest.xml with the correct intent filter so FCM knows to route messages to it.

```
// MyFirebaseMessagingService.kt

@AndroidEntryPoint  // Hilt injection into a Service
class MyFirebaseMessagingService : FirebaseMessagingService() {

    @Inject lateinit var userRepository: UserRepository  // Save token to backend

    // Called when FCM assigns a NEW token to this app instance.
    // Critical: send this token to your server so it can address messages to this device.
    override fun onNewToken(token: String) {
        super.onNewToken(token)
        // Launch coroutine to send token to your backend API
        CoroutineScope(Dispatchers.IO).launch {
            userRepository.updateFcmToken(token)
        }
    }

    // Called when a message arrives and the app is in the FOREGROUND.
    // Also called for data-only messages in any app state.
    override fun onMessageReceived(message: RemoteMessage) {
        super.onMessageReceived(message)

        // message.notification — present if server sent a notification payload
        // message.data         — present if server sent a data payload

        val title = message.notification?.title          // e.g. "Order Shipped!"
            ?: message.data["title"]                    // Fallback to data payload
        val body  = message.notification?.body           // e.g. "Your order #1234..."
            ?: message.data["body"]

        if (title != null && body != null) {
            showNotification(title, body, message.data)
        }
    }
```

Step 2 — Building and Displaying the Notification

```
    private fun showNotification(title: String, body: String, data: Map<String, String>) {

        // Build an Intent that opens OrderDetailActivity when notification is tapped.
        // data["orderId"] comes from the server's data payload.
        val intent = Intent(this, OrderDetailActivity::class.java).apply {
            putExtra("orderId", data["orderId"])  // Pass order ID to destination screen
            // FLAG_ACTIVITY_CLEAR_TOP: if OrderDetailActivity exists in back stack, clear above it
            flags = Intent.FLAG_ACTIVITY_CLEAR_TOP or Intent.FLAG_ACTIVITY_SINGLE_TOP
        }

        // PendingIntent wraps the Intent so the system can fire it later (when user taps).
        // FLAG_IMMUTABLE required on Android 12+ for security reasons.
        val pendingIntent = PendingIntent.getActivity(
            this, 0, intent,
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )

        // Build the notification using NotificationCompat for backward compatibility.
        val notification = NotificationCompat.Builder(this, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_notification)  // Must be a monochrome icon
            .setContentTitle(title)                    // Bold first line in notification tray
            .setContentText(body)                      // Second line
            .setStyle(NotificationCompat.BigTextStyle().bigText(body)) // Expand on long press
            .setPriority(NotificationCompat.PRIORITY_HIGH) // Heads-up notification
            .setAutoCancel(true)          // Dismiss when user taps
            .setContentIntent(pendingIntent)           // What happens on tap
            .build()

        // NotificationManager posts the notification to the system tray.
        // notificationId is unique per notification type — reusing ID updates existing one.
        val manager = getSystemService(NOTIFICATION_SERVICE) as NotificationManager
        manager.notify(NOTIFICATION_ID, notification)
    }

    companion object {
        const val CHANNEL_ID = "order_updates"  // Must match channel created in Application
        const val NOTIFICATION_ID = 1001
    }
}
```

Step 3 — AndroidManifest.xml Registration

The service MUST be declared in the manifest with the RECEIVE intent filter. Without this, FCM does not know which component to deliver messages to.

```
<!-- AndroidManifest.xml -->
<service
    android:name=".MyFirebaseMessagingService"
    android:exported="false">  <!-- false = only FCM can call this service -->
    <intent-filter>
        <!-- This exact action is required — it tells FCM to route here -->
        <action android:name="com.google.firebase.MESSAGING_EVENT"/>
    </intent-filter>
</service>

<!-- Android 13+ notification permission -->
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
```

Step 4 — Notification Channel Setup (Application class)

Channels must be created before any notification can be posted. The Application class onCreate() runs before any Activity, making it the safest place for channel creation.

```
class MyApplication : Application() {

    override fun onCreate() {
        super.onCreate()
        createNotificationChannels()
    }

    private fun createNotificationChannels() {
        // Channels only apply on Android 8.0 (API 26) and above
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {

            val channel = NotificationChannel(
                "order_updates",           // Channel ID — must match CHANNEL_ID in service
                "Order Updates",           // User-visible name shown in Settings
                NotificationManager.IMPORTANCE_HIGH  // Enables heads-up notification
            ).apply {
                description = "Notifications about your order status"
                enableLights(true)            // LED indicator
                lightColor = Color.GREEN
                enableVibration(true)
            }

            val manager = getSystemService(NotificationManager::class.java)
            manager.createNotificationChannel(channel)
        }
    }
}
```

Step 5 — Server-Side: Sending the Message

This is the Node.js server code (using Firebase Admin SDK) that sends the notification when an order status changes to 'shipped'.

```
// server.js (Node.js + Firebase Admin SDK)
const admin = require('firebase-admin');

// Initialize once — uses service account credentials
admin.initializeApp({
  credential: admin.credential.applicationDefault()
});

async function sendOrderShippedNotification(fcmToken, orderId, trackingNumber) {
  const message = {
    token: fcmToken,       // The registration token saved from onNewToken()

    // notification block — auto-displayed by FCM when app is in background
    notification: {
      title: 'Order Shipped! ■',
      body: `Your order #${orderId} is on its way. Track: ${trackingNumber}`,
    },

    // data block — always delivered to onMessageReceived()
    data: {
      orderId: orderId,          // Passed to OrderDetailActivity via Intent extra
      trackingNumber: trackingNumber,
      type: 'ORDER_SHIPPED',     // Lets app route to correct screen
    },

    // Android-specific overrides
    android: {
      priority: 'high',          // Wake device even in doze mode
      notification: {
        channelId: 'order_updates',  // Must match channel created in app
        icon: 'ic_notification',     // Drawable resource name in app
        color: '#FF6D00',            // Notification accent color
        clickAction: 'OPEN_ORDER_DETAIL',  // Optional: intent action override
      },
    },
  };

  // Send and get message ID for logging/debugging
  const response = await admin.messaging().send(message);
  console.log('Message sent successfully:', response);
  return response;
}
```

Step 6 — Reading FCM Data in the Destination Activity

```
// OrderDetailActivity.kt
class OrderDetailActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // When notification is tapped, FCM data payload arrives here as extras.
        // This works for both notification+data and pure data messages.
        val orderId = intent.getStringExtra("orderId")

        orderId?.let {
            viewModel.loadOrder(it)  // Fetch order details using the ID
        }
    }

    // If app was already open and receives a new notification tap:
    override fun onNewIntent(intent: Intent) {
        super.onNewIntent(intent)
        val orderId = intent.getStringExtra("orderId")
        orderId?.let { viewModel.loadOrder(it) }
    }
}
```

Complete Flow Summary

Stage	Who Acts	What Happens
Token Registration	FCM SDK + Your App	onNewToken() fires; app sends token to your backend
Order Ships	Your Backend Server	Admin SDK sends HTTP v1 request to FCM with token + payload
FCM Routes	Google FCM Servers	Message queued/delivered to device via Play Services connection
App Foreground	onMessageReceived()	Your code calls showNotification() — displayed with PendingIntent
App Background	FCM SDK	Auto-displayed; data payload in Intent extras on tap
User Taps	Android OS	PendingIntent fires, OrderDetailActivity opens with orderId extra

PRO TIP
Always send both notification AND data payloads together. The notification payload handles auto-display when backgrounded, while the data payload ensures your app always receives the structured information it needs to act on the message.

6 Quick Reference Cheat Sheet

FCM HTTP v1 API Endpoint

```
POST https://fcm.googleapis.com/v1/projects/{PROJECT_ID}/messages:send
Authorization: Bearer {OAUTH2_ACCESS_TOKEN}
Content-Type: application/json
```

Message Size Limits

Message Type	Max Size
Notification payload	Not strictly limited, keep under 1KB
Data payload	4 KB total
Combined (notification + data)	4 KB total

Common Troubleshooting

Issue	Likely Cause & Fix
Notification not shown in background	Missing or wrong channel ID on Android 8+; check CHANNEL_ID matches
onMessageReceived not called	App in background + notification payload; FCM SDK intercepts it
Token is null	google-services.json missing or misplaced; check app/google-services.json
InvalidRegistration error from server	Stale token; implement onNewToken() and update your server
Notification not shown on Android 13	POST_NOTIFICATIONS permission not granted; request at runtime
Message not delivered	TTL expired; device offline longer than TTL value (default 4 weeks)